CHAPTER 2: LITERATURE REVIEW

2.1 Introduction

This will be a review based on all the studies collected on mobile application vulnerabilities. Mobile applications have been the norm and most importantly, the go-to endpoint for activities in your everyday life such as healthcare, digital banking and e-governance. The vast growth of mobile application reliance has been accompanied by the growth of attack surfaces. All 40 of the studies were analyzed to compare vulnerabilities, ways to mitigate, and ways to detect.

2.1.1 *Background of the Study*

The studies address the vast growth of mobile application use in important operations. This is because it has become one of the most convenient ways for them to operate. Operations such as healthcare management have transitioned most of the processes into mobile platforms to add efficiency and accessibility. Though the transition also means an opportunity for attack surfaces to arise.

2.1.2 Scope of Reviewed Literature

Four themes have been deduced after thorough evaluation which are Automated Vulnerability Assessment and Intelligent Threat Detection, Sector-Specific Security Postures and Regulatory Compliance, Cross-Platform Ecosystems, and Mobile Cloud Computing (MCC) Architectural Security.

2.2 Thematic Review of Mobile Application Security

This is a review of all the four themes found within the 40 studies. It would help find out the research gaps.

2.2.1 *Automated Vulnerability Assessment and Intelligent Threat Detection*

Automated vulnerability assessment is the combination of Static Application Security Testing (SAST), Dynamic Application Security Testing (DAST) and Machine Learning is the trifecta for auditing mobile codebases and being able to detect emerging threats. Static scanning tools are the foundation for code auditing (Chandra et al., 2023; Kusreynada & Barkah, 2024). The recent rise of tools such as SaveVision (Mittal et al., 2026) and ScanDroidX (Anthony et al., 2026) help get analysis on cryptographic analysis and MITRE ATT&CK mapping, providing more coverage. The gaining occurrence in researchers reaching for integrated SAST/DAST pipelines that use Frida and Appium (Swamy et al., 2025; Nagarajan et al., 2025) when they want to validate dynamic behaviors has also been a new topic. ML algorithms such as Multilayer Perceptron (MLP) (Kushwaha et al., 2023), Random Forest (Singh et al., 2025), and CMC permission reduction (Jain et al., 2023) are used to find zero-day malware. MLSTC (Kumari & Sharma, 2023) were proposed to check network packets. It acted as the door before letting the packets in so that the operating system can execute them. A problem with static analyzers is the wide range in performance , ranging from 22% to 67% (Zhu et al., 2024). This led to the rise of new more effective methods such as hybrid pipelines (Swamy et al., 2025; Nagarajan et al., 2025) and dynamic instrumentation (Kusreynada & Barkah, 2024) which have successfully found runtime flaws such as SSL pinning bypasses. ML-based classifiers have great detection accuracy with models that have reached 99.40% accuracy, the problem is that the accuracy is based on clean datasets as opposed to real-world malware (Zhu et al., 2024; Cinar & Kara, 2023). The MLSTC chips have remained as prototypes with no signs of official deployment. The pattern shows that research in mobile defenses is neglected with the research gap of the need of dynamic testing pipelines that are built into the CI/CD workflows.

2.2.2 *Sector-Specific Security Postures and Regulatory Compliance*

Multiple operations have relied on mobile applications, even with their worrying security posture. Several healthcare applications were included in this study. Indonesia's PeduliLindungi and Mobile JKN have showed frequent OWASP vulnerabilities ranging from deprecated cryptographic hashes to exposed API keys (Priambodo et al., 2022). Cao et al. (2025) did a huge number by reviewing 213 healthcare applications which showed that only 75.9% of the apps had confidentiality mechanisms, 37.4% of the apps were lacking in data integrity and 62.7% of the apps had poor system availability. Abdullah et al. (2024) provided a solution for secure Electronic Health Records (EHR) by having a lightweight AES-256 and Elliptic Curve Cryptography (ECC) hybrid model. Zhu and Chen (2022) proposed multi-level access control for QR in the medical department. Financial and healthcare prioritize custom SDK controls (Kothari et al., 2024), RASP (Blanco-Muñoz et al., 2024) and hybrid AES-ECC encryption (Abdullah et al., 2024). Although they still lack in preventing permission manipulation and have weak authorization logic (Mykhaylova et al., 2023; Al-Hakimi et al., 2025). The studies go between post-hoc dynamic audits (Falade & Ogundele, 2023; Al-Delayel, 2022) and proactive security frameworks (Al-Sherideh et al., 2024; Chowdhury et al., 2025). The studies have shown the systematic vulnerabilities throughout commercial applications but do suffer from basing the results on a small sample size (Al-Hakimi et al., 2025; Priambodo et al., 2022). Also, the preventative measures are mostly reactive instead of having them implemented during development (Chandra et al., 2023; Abdullah & Zeebaree, 2021). There is common occurrence of basic cryptographic and storage errors that would fail the regulatory guideline. This makes them lack security, making a gap for Secure Software Development Life Cycle (SSDLC) frameworks.

2.2.3 *Cross-Platform Ecosystems, Network Transport, and User Privacy*

There has been rapid growth in cross-platform development frameworks. Although they do come with insecure network transport channels that give way to new attack vectors. Cross. Fedynyshyn et al. (2024) analysed 6165 APKs which showed that Xamarin apps had the most binary flaws and React Native had the most code-level vulnerabilities. Rambhia et al. (2023) had a counter towards the claim that Flutter's compilation blocks tampering by proving that Flutter binaries were vulnerable to dynamic memory manipulation. The only way to counter this is by explicitly implementing the security measures beforehand. The binary analysis has shown that Xamarin applications have a high vulnerability rate, whereas React Native apps have high source-code permissions (Fedynyshyn et al., 2024). PrivProt shows the minimal latency (Cevallos-Salas et al., 2025), but audits Sakuraba et al. (2025) say that commercial apps fail to provide basic encryption. The research gap is the amount of disconnect between the designated privacy policies and the actual runtime network behaviour. It shows the need for automated verification gateways that would then validate the network traffic.

2.2.4 *Mobile Cloud Computing (MCC) and Architectural Security*

Mobile Cloud Computing would offload computational tasks to the backend infrastructures which would then solve the hardware limitations issue but also open the gate to complex ecosystem vulnerabilities (Sheikh et al., 2023; Alqarqaz & Younes, 2022). Tang et al. (2022) created vTrust which is a remote Trusted Execution Environment (TEE). Within a remote cloud virtual machine, it processes the application, uses a local hypervisor that encrypts device screen frames and touch inputs (AES-128-GCM), and blinds the compromised OS (Tang et al., 2022). Tandri et al. (2022) showed how efficient the cloud offloading was by running a resource-heavy sign-language translation app using Google App Engine (PaaS). It reduces the clients' CPU utilization to under 2%, yet the latency was just 6.07 ms. vTrust (Tang et al., 2022) and PaaS API designs (Tandri et al., 2022) have solved the resource limitations. On the other end, multi-tenant app-in-app ecosystems (Zhang et al., 2024) have the opposite results by proving that sharing resources between host apps give severe cross-party impersonation risks. Cloud offloading solutions have shown great proof of good performance by reducing the client CPU utilization to under 2% with low latency (Tandri et al., 2022). Although a problem with MCC literature is that it's tested in controlled environments (Tang et al., 2022; Tandri et al., 2022) or theoretical taxonomies (Chimuco et al., 2023; Sheikh et al., 2023; Alqarqaz & Younes, 2022) without actual proof of it reducing latency in 6G network conditions. The findings show that mobile applications are evolving. Traditional single-device security models have become a thing of the past, meaning the need for multi-party Zero Trust authorization models in the future.

2.3 *Summary of Literature and Research Implications*

2.3.1 *Thematic Synthesis Table*

Theme	Supporting Studies	Common Dataset / Data Source	Methods / Approaches	Key Findings / Trends	Limitations / Research Gap
1. Automated Detection & AI Threat	(Zhu et al., 2024; Kusreynada & Barkah, 2024; Mittal et al., 2026;	Google Play, GHERA, MSTG, CICMalDroid2020, VirusTotal	SAST/DAST testing, ML classifiers, CMC feature ranking, and MLSTC hardware	Individual SAST tools have bad coverage of 22%–67% (Zhu et al., 2024). Hybrid pipelines can find threats (Swamy et al.,	The industry relies on static analysis, not paying attention to payloads after installation (Zhu et al., 2024;

Theme	Supporting Studies	Common Dataset / Data Source	Methods / Approaches	Key Findings / Trends	Limitations / Research Gap
Intelligence	Singh et al., 2025; Swamy et al., 2025; Cinar & Kara, 2023; Nagarajan et al., 2025; Anthony et al., 2026; Kushwaha et al., 2023; Jain et al., 2023; Kumari & Sharma, 2023)			2025; Anthony et al., 2026). ML models (MLP) have max 99.4% accuracy against zero-day malware (Kushwaha et al., 2023).	Cinar & Kara, 2023).
2. Sector-Specific Postures & Compliance	(Chandra et al., 2023; Mykhaylova et al., 2023; Al-Sherideh et al., 2024; Falade & Ogundele, 2023; Al-Delayel, 2022; Al-Hakimi et al., 2025; Zhu & Chen, 2022; Blanco-Muñoz et al., 2024; Bardhan et al., 2024; Chowdhury et al., 2025; Abdullah et al., 2024; Priambodo et al., 2022; Cao et al., 2025;	mHealth platforms, banking software, and e-government portals	Dynamic code audits, SQLite extraction, Securix SDK enforcement, and RASP integrations	Used hashes and plaintext are just floating within the platform (Bardhan et al., 2024; Priambodo et al., 2022). SDKs have rules against screenshots (Kothari et al., 2024).	Security controls are given after deployment rather than before, during the development phase (Mykhaylova et al., 2023). Empirical sector audits use small sample sizes (Al-Hakimi et al., 2025).

Theme	Supporting Studies	Common Dataset / Data Source	Methods / Approaches	Key Findings / Trends	Limitations / Research Gap
	Kothari et al., 2024)				
3. Cross-Platform Ecosystems & Network Privacy	(Cevallos-Salas et al., 2025; Sakuraba et al., 2025; Shah et al., 2024; Nikkhah et al., 2024; Encalada et al., 2022; Fedynyshyn et al., 2024; Rambhia et al., 2023)	12,426 top Google Play apps, 6,165 cross-platform APKs, and surveys.	Traffic interception, HWRSM risk scoring, framework binary analysis, and Privacy Calculus modelling	Frameowrks that weren't built within the platform give new risks (Fedynyshyn et al., 2024). 65 top apps showed data in plaintext even though 46 claimed good encryption (Sakuraba et al., 2025). PrivProt lowered transit latency to 23.08 ms (Cevallos-Salas et al., 2025).	Automated scanners have a hard time scanning some frameworks (Fedynyshyn et al., 2024). App store privacy doesn't prove itself with actual network behavior (Sakuraba et al., 2025).
4. Cloud Computing & Architectural Security	(Tang et al., 2022; Zhang et al., 2024; Chimuco et al., 2023; Khan, 2022; Sheikh et al., 2023; Abdullah & Zeebaree, 2021; Tandri et al., 2022; Alqarqaz & Younes, 2022)	9 major app-in-app ecosystems, Google App Engine, and hypervisors	Remote Trusted Execution Environments, PaaS workload offloading, and cross-party authorization	PaaS offloading ensures client-side CPU is under 2% (Tandri et al., 2022). Remote TEEs separate the application from compromised systems (Tang et al., 2022). Super-apps show major authorization risks (Zhang et al., 2024).	Multiple authorization boundaries are still too complicated (Zhang et al., 2024). The study relies on theory rather than testing (Sheikh et al., 2023).

2.3.2 *Identified Research Gaps*

a. **Research Gap 1**. Limited integration of static and dynamic security testing. Many studies rely heavily on Static Application Security Testing (SAST), which is effective for analysing application code, permissions and configurations but still may fail to detect vulnerabilities that appear only during runtime. The reviewed studies also show differences in the coverage of individual SAST tools, while hybrid approaches combining static and dynamic analysis provide better detection of runtime threats. However, the integration of these techniques inti standard development workflows remains limited.

b. **Research Gap 2.** Limited real world and diverse datasets. Several studies use small datasets, selected applications or controlled testing environments. Specific sector of studies also focuses on applications from limited geographic locations or specific categories. As a result, the findings may not fully represent the diversity of vulnerabilities found across the wider mobile application ecosystem.

c. **Research Gap 3.** Limited detection of evolving threats. Machine learning based approaches have achieved high detection accuracy. However, its performance is often evaluated using existing and controlled datasets. This creates uncertainty of its ability to identify previously unseen malware and new attack techniques. Static analysis may also fail to detect malicious behaviour that appears only after installation or during execution.

d. **Research Gap 4.** Security limitations in cross platform applications. The growing use of frameworks such as Flutter, React Native and Xamarin introduces framework specific attack vectors. The reviewed studies indicate that automated security tools may struggle to accurately analyse intermediate binaries and runtime behaviour in cross platform applications.